



Security Review For SuperReturn



Collaborative Audit Prepared For:
Lead Security Expert(s):

SuperReturn
ctf_sec
Kirkeelee

Date Audited:

November 14 - November 21, 2025

Introduction

SuperReturn is the most transparent earn platform that generates a sustainable yield using both onchain and RWA sources. This audit focuses on some UX improvements that will be deployed across all EVM networks supported by SuperUSD and sSuperUSD.

Scope

Repository: SuperReturn/superreturn-superUSD

Audited Commit: c3a1884c40d692303ad52de23a66ce900646e98d

Final Commit: f4dc41334a437e8ab30b4fb0ce63bf980dfb5147

Files:

- src/atomic-queue/AtomicQueue.sol
- src/atomic-queue/AtomicSolverV4.sol
- src/base/BoringVault.sol
- src/base/Roles/AccountantWithRateProviders2.sol
- src/base/Roles/AccountantWithRateProviders.sol

Final Commit Hash

f4dc41334a437e8ab30b4fb0ce63bf980dfb5147

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	2	3

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Cross chain rate/value mismatch allowing value extraction. [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/issues/6>

Summary

Bridge mints/burns raw share amounts while each chain uses its own local `exchangeRate`, so moving shares across chains can change the economic value of the same share amount.

Vulnerability Detail

`crosschainMint` / `crosschainBurn` operate on share counts without consulting the accountant's `exchangeRate` or converting value between chains. If `exchangeRate` differs between chains, 1 share on chain A != 1 share on chain B in economic terms, enabling arbitrage by moving shares across chains.

```
/// @notice Allows the SuperchainTokenBridge to mint tokens.
/// @param _to Address to mint tokens to.
/// @param _amount Amount of tokens to mint.
function crosschainMint(address _to, uint256 _amount) external {
    if (msg.sender != 0x4200000000000000000000000000000000000000000000000000000000000000) revert
        ↪ NotSuperchainERC20Bridge();

    _mint(_to, _amount);

    emit CrosschainMint(_to, _amount, msg.sender);
}

/// @notice Allows the SuperchainTokenBridge to burn tokens.
/// @param _from Address to burn tokens from.
/// @param _amount Amount of tokens to burn.
function crosschainBurn(address _from, uint256 _amount) external {
    if (msg.sender != 0x4200000000000000000000000000000000000000000000000000000000000000) revert
        ↪ NotSuperchainERC20Bridge();

    _burn(_from, _amount);

    emit CrosschainBurn(_from, _amount, msg.sender);
}
```

Impact

An attacker can move shares from a low-rate chain to a high-rate chain and realize the value difference; loss scales with volume and can be large.

Code Snippet

<https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/blob/dffe893e9b0398d690ed9ce0efd712ca8d89258a/superreturn-superUSD/src/base/BoringVault.sol#L176-L182>

Tool Used

Manual Review

Recommendation

The team confirmed that the crosschain functions will be removed. Alternatively, Mint/burn based on value, not raw shares: when minting on target chain compute shares = `value_burned_in_base / local_exchangeRate`.

Discussion

ctf-sec

<https://github.com/SuperReturn/superreturn-superUSD/pull/8>

Cross-chain transfer feature is removed.

Issue M-1: `instantWithdraw` DoS from per want liquidity check. [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/issues/8>

Summary

`instantWithdraw` converts all pending withdrawals into the requested want token and compares to `vault.balanceOf(want)`, which will frequently fail for a vault that holds multiple asset types.

Vulnerability Detail

`withdrawInProgressAmount` aggregates pending shares across the vault. The code converts that aggregate to the requested want and compares to the vault's on-chain balance of that single want token; when the vault is diversified the want balance is typically smaller than the converted total, causing `instantWithdraw` flow to revert.

Impact

DoS of `instantWithdraw` flow.

Code Snippet

<https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/blob/dffe893e9b0398d690ed9ce0efd712ca8d89258a/superreturn-superUSD/src/atomic-queue/AtomicQueue.sol#L532-L544>

Tool Used

Manual Review

Recommendation

Track pending shares per (offer, want) pair and compare only that per-want bucket (converted to want units) against `vault.balanceOf(want)`.

Discussion

ctf-sec

```
/**
 * @notice This mapping tracks the total amount of 'offer' tokens in progress for
 * ↪ withdrawal requests per want token.
 * @dev It is incremented when a new request is submitted and decremented when the
 * ↪ request is fulfilled or cancelled.
 */
mapping(address => mapping(address => uint256)) public withdrawInProgressAmount;
```

<https://github.com/SuperReturn/superreturn-superUSD/pull/10>

The fix track withdrawInProgressAmount per offer token per want token and avoid DOS in multi asset case.

Issue M-2: Share transfer timelock can be bypassed via cross-chain transfer in BoringVault.sol [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/issues/10>

Summary

Share transfer timelock can be bypassed via cross-chain transfer

Vulnerability Detail

Impact

When the BoringVault.sol transferFrom function triggers a hook check to apply the timelock.

However, user can bypass this timelock by sending cross-chain asset because the _mint and burn function does not apply the hook timelock check.

```
function crosschainMint(address _to, uint256 _amount) external {
    if (msg.sender != 0x4200000000000000000000000000000000000000000000000000000000000000) revert
    ↪ NotSuperchainERC20Bridge();

    _mint(_to, _amount);

    emit CrosschainMint(_to, _amount, msg.sender);
}

/// @notice Allows the SuperchainTokenBridge to burn tokens.
/// @param _from Address to burn tokens from.
/// @param _amount Amount of tokens to burn.
function crosschainBurn(address _from, uint256 _amount) external {
    if (msg.sender != 0x4200000000000000000000000000000000000000000000000000000000000000) revert
    ↪ NotSuperchainERC20Bridge();

    _burn(_from, _amount);

    emit CrosschainBurn(_from, _amount, msg.sender);
}
```

Code Snippet

<https://github.com/ethereum-optimism/optimism/blob/5b965845530146cff6e0bb3390d4e3fa203925c7/packages/contracts-bedrock/src/L2/SuperchainTokenBridge.sol#L72>

<https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/blob/dffe893e9b0398d690ed9ce0efd712ca8d89258a/superreturn-superUSD/src/base/BoringVault.sol#L167>
<https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/blob/dffe893e9b0398d690ed9ce0efd712ca8d89258a/superreturn-superUSD/src/base/Roles/TellerWithMultiAssetSupport.sol#L189>

Tool Used

Manual Review

Recommendation

Apply the hook timelock check before `_mint` and burn in `BoringVault.sol`

Discussion

ctf-sec

<https://github.com/SuperReturn/superreturn-superUSD/pull/8/files>

The cross-chain feature is removed to fix the issue.

Issue L-1: `maxTotalSupply` bypass via `crosschainMint`. [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/issues/7>

Summary

The bridge mint function bypasses the vault's `maxTotalSupply` check and can inflate supply above the intended cap.

Vulnerability Detail

`BoringVault` exposes `maxTotalSupply` but `crosschainMint` mints shares without validating `totalSupply + _amount <= maxTotalSupply`, so bridge calls can increase supply beyond the cap.

Impact

Breaks supply invariants and enables inflation.

Code Snippet

<https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/blob/dffe893e9b0398d690ed9ce0efd712ca8d89258a/superreturn-superUSD/src/base/BoringVault.sol#L111-L114> <https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/blob/dffe893e9b0398d690ed9ce0efd712ca8d89258a/superreturn-superUSD/src/base/BoringVault.sol#L176-L182>

Tool Used

Manual Review

Recommendation

Team plans to deprecate the bridge functionality and it will fix the issue without complex solutions.

Discussion

ctf-sec

<https://github.com/SuperReturn/superreturn-superUSD/pull/8/files>

Issue L-2: Unnecessary ERC20 approvals before teller.bulkWithdraw [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/issues/9>

Summary

AtomicQueue calls `offer.safeApprove(address(teller), 0)` and then `offer.safeApprove(address(teller), offerAmount)` immediately before `teller.bulkWithdraw`, but the teller ultimately calls `vault.exit` which uses an internal `_burn` (no ERC20 allowance consumed), so these approvals are unnecessary.

Vulnerability Detail

The approvals are performed even though the teller does not rely on `ERC20.allowance/transferFrom` to burn shares, it burns via the vault's `exit` (which updates vault accounting directly). The approvals therefore waste gas and add a minor unnecessary on-chain action.

Impact

Unnecessary gas usage.

Code Snippet

<https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/blob/dffe893e9b0398d690ed9ce0efd712ca8d89258a/superreturn-superUSD/src/atomic-queue/AtomicQueue.sol#L554-L557>

Tool Used

Manual Review

Recommendation

Delete two approvals.

Discussion

ctf-sec

Fix looks good.

Issue L-3: accountant.getRateInQuoteSafe can be cached [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/issues/11>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

accountant.getRateInQuoteSafe can be cached to save gas.

Vulnerability Detail

The value accountant.getRateInQuoteSafe(want) is called twice but the returned can be cached.

```
// Calculate assets that will be withdrawn
assetsOut = offerAmount.mulDivDown(
    accountant.getRateInQuoteSafe(want),
    ONE_SHARE
);

uint256 assetOutWithDiscount = assetsOut.mulDivDown(DISCOUNT_DENOMINATOR -
    ↪ discount, DISCOUNT_DENOMINATOR);

// Calculate assets needed for pending withdrawal requests
uint256 pendingWithdrawAssets =
    ↪ withdrawInProgressAmount[address(offer)].mulDivDown(
        accountant.getRateInQuoteSafe(want),
        ONE_SHARE
    );
```

Impact

Gas optimization.

Code Snippet

<https://github.com/sherlock-audit/2025-11-sake-finance-nov-12th/blob/dffe893e9b0398d690ed9ce0efd712ca8d89258a/superreturn-superUSD/src/atomic-queue/AtomicQueue.sol#L525>

Tool Used

Manual Review

Recommendation

```
uint256 safeRate = accountant.getRateInQuoteSafe(want);
// Calculate assets that will be withdrawn
assetsOut = offerAmount.mulDivDown(
    safeRate,
    ONE_SHARE
);

uint256 assetOutWithDiscount = assetsOut.mulDivDown(DISCOUNT_DENOMINATOR -
    ↪ discount, DISCOUNT_DENOMINATOR);

// Calculate assets needed for pending withdrawal requests
uint256 pendingWithdrawAssets =
    ↪ withdrawInProgressAmount[address(offer)].mulDivDown(
        safeRate,
        ONE_SHARE
    );
```

Discussion

ctf-sec

Just a gas optimization. No fix.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.